

VLSI DESIGN INTERNSHIP – TASK 4 ON RTL DESIGN OF FINITE STATE MACHINES (FSM) AND CONTROL UNITS



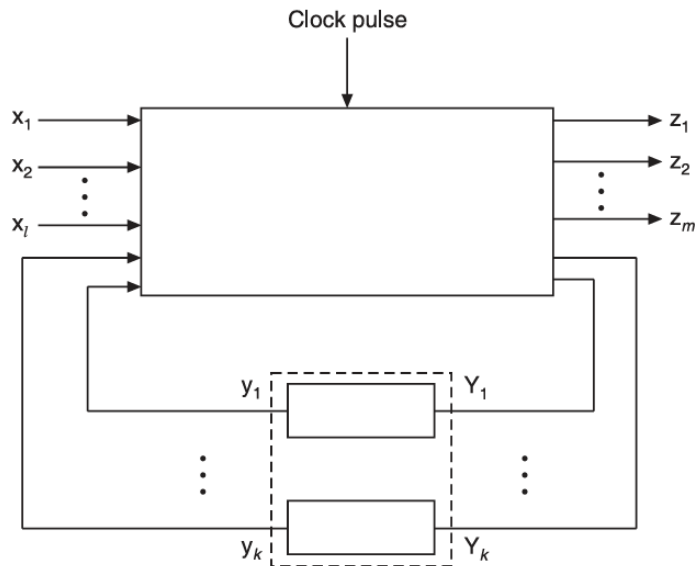
Submitted To :
Maincrafts Technology

Submitted By :-
Name - Ankit Kumar
Roll Number – 24ECE006
College Name - Ganga Institute of
Technology and Management, Haryana

Finite State machine

A finite state machine is an abstract model that describes the synchronous sequential machine. Since in a sequential circuit the output depends on the present input as well as on the past inputs.

Block Diagram of FSM :



Some terms related to FSM :

1. **state diagram** - A state diagram is a graphical representation of an FSM (Finite State Machine).
2. **state table** - A state table is a table that shows how an FSM changes from its present state to its next state based on the input.
3. **state equation** - A state equation is a Boolean equation that tells us what the next state will be from the present state and the inputs.

There are two types of FSM :-

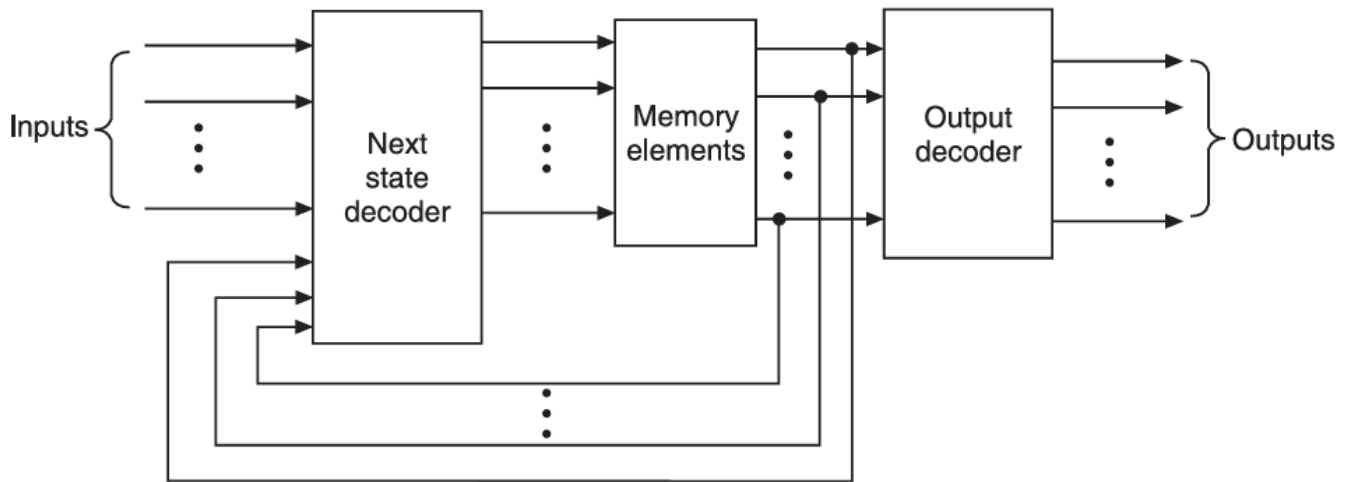
- (i) Moore FSM
- (ii) Mealy FSM

Moore FSM :

when the output of the sequential circuit depends only on the present state of the flip-flop, the sequential circuit is referred to as the Moore circuit or the Moore machine.

Output = $f(\text{present state})$

Moore FSM circuit model :

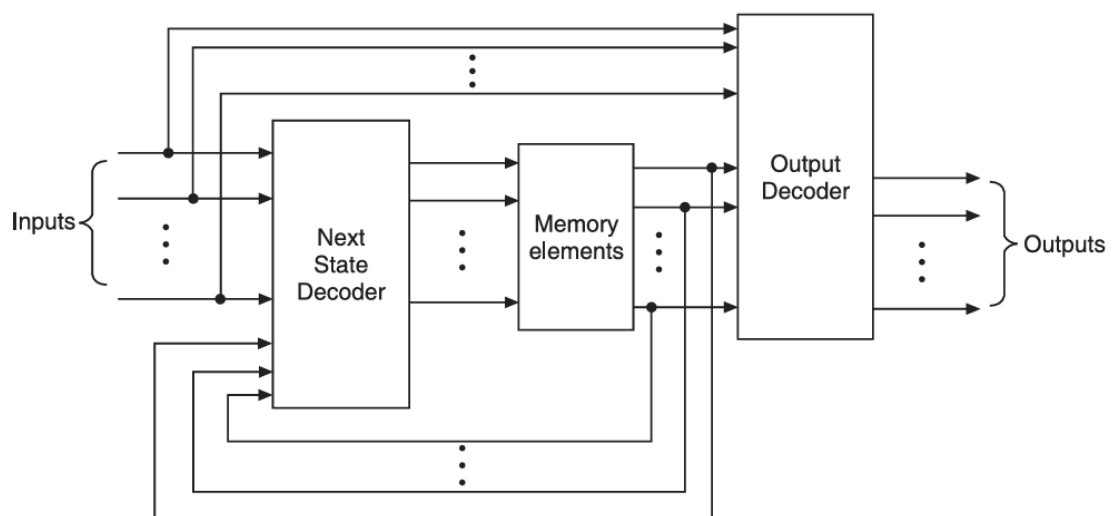


Mealy FSM :

When the output of the sequential circuit depends on both the present state of the flip-flops and on the inputs, the sequential circuit is referred to as Mealy circuit or Mealy machine.

Output = $f(\text{present state, input})$

Mealy FSM circuit Model :



Steps to design FSM :-

- Understand the problem statement
- Identify the states
- Draw the State Diagram
- Make the State Transition Table
- If redundant states present then remove that
- Choose State Encoding
- Derive the State Equations / Next-State Logic
- Design the Output Logic
- Design Logic circuit
- Write the Verilog Code
- Write the Testbench
- Simulate and Check the Waveform

Problem 1. Design a simple FSM that cycles through 4 states .

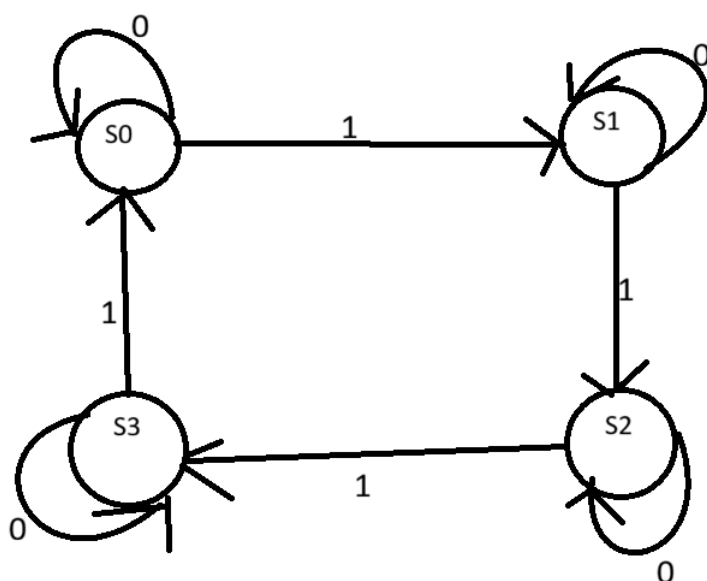
Solution –

States – s0 - s1 - s2 - s3 - s0 - s1 - s2.....
00 – 01 – 10 – 11 – 00 – 01 – 10

Input – none

Output – Binary value of current state

State Transition Diagram :



State Table :

Present State	Current State		Next State		Flip flop Inputs	
	Q1	Q0	Q1+	Q0+	D1	D0
S0	0	0	0	1	0	1
S1	0	1	1	0	1	0
S2	1	0	1	1	1	1
S3	1	1	0	0	0	0

Logic Equation –

$$D0 = f(Q0, Q1)$$

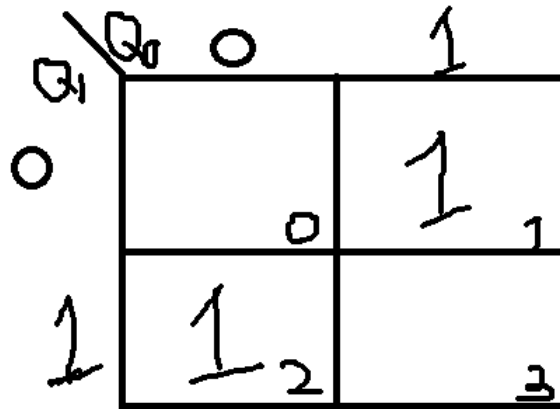
$$D1 = g(Q0, Q1)$$

$$\text{Output} = h(Q0, Q1)$$

$$D1 = \sum m(1, 2)$$

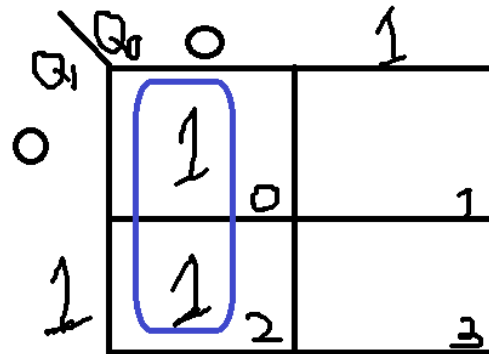
$$D0 = \sum m(0, 2)$$

K map of D1:



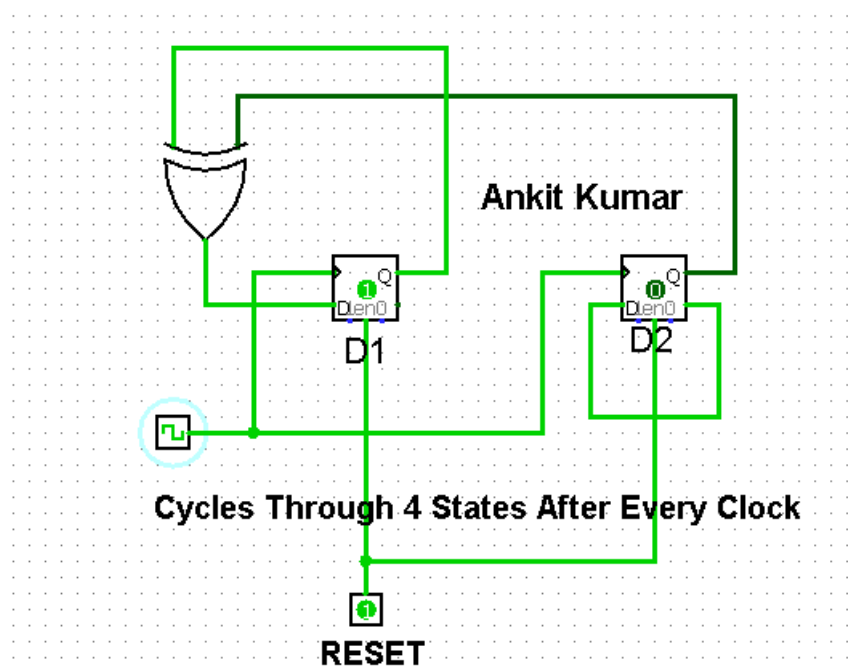
$$D1 = Q0 \oplus Q1$$

K map for D0 :



$$D0 = Q0'$$

Digital Logic Circuit Diagram :-



Verilog : fsm_4state

```
module fsm_4state (  
    input clk,  
    input reset,  
    output reg [1:0] y  
);  
  
    reg [1:0] state;  
  
    // State encoding  
    parameter S0 = 2'b00;  
    parameter S1 = 2'b01;  
    parameter S2 = 2'b10;  
    parameter S3 = 2'b11;  
  
    // State register  
    always @(posedge clk) begin  
        if (reset)  
            state <= S0;  
        else begin  
            case (state)  
                S0: state <= S1;  
                S1: state <= S2;  
                S2: state <= S3;  
                S3: state <= S0;  
                default: state <= S0;  
            endcase  
        end  
    end  
  
    // Moore output logic  
    always @(*) begin  
        case (state)  
            S0: y = 2'b00;  
            S1: y = 2'b01;  
            S2: y = 2'b10;  
            S3: y = 2'b11;  
            default: y = 2'b00;  
        endcase  
    end  
  
endmodule
```

Verilog testbench : fsm_4state

```
`timescale 1ns/1ps

module tb_fsm_4state;

    reg clk;
    reg reset;
    wire [1:0] y;

    // Instantiate DUT (Device Under Test)
    fsm_4state uut (
        .clk(clk),
        .reset(reset),
        .y(y)
    );
    // Clock generation: 10 ns period
    always #5 clk = ~clk;

    initial begin
        // Initialize
        clk = 0;
        reset = 1;

        // Keep reset active for 10 ns
        #10;

        // Release reset
        reset = 0;

        // Run for 50 ns
        #50;

        $finish;
    end

    // Display values whenever they change
    initial begin
        $monitor("Time = %0t | Reset = %b | State = %b | Output y = %b",
            $time, reset, uut.state, y);
    end

    // Generate waveform
    initial begin
        $dumpfile("fsm_4state.vcd");
        $dumpvars(0, tb_fsm_4state);
    end
endmodule
```


Output :

```
VCD info: dumpfile fsm_4state.vcd opened for output.  
Time = 0 | Reset = 1 | State = xx | Output y = xx  
Time = 5000 | Reset = 1 | State = 00 | Output y = 00  
Time = 10000 | Reset = 0 | State = 00 | Output y = 00  
Time = 15000 | Reset = 0 | State = 01 | Output y = 01  
Time = 25000 | Reset = 0 | State = 10 | Output y = 10  
Time = 35000 | Reset = 0 | State = 11 | Output y = 11  
Time = 45000 | Reset = 0 | State = 00 | Output y = 00  
Time = 55000 | Reset = 0 | State = 01 | Output y = 01  
testbench.sv:35: $finish called at 60000 (1ps)
```

Waveform :



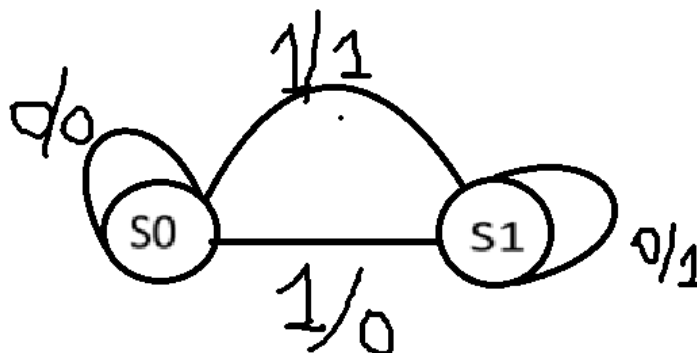
Problem 2: Output becomes high when input = 1 and current state is S1.

solution : Input – 0,1

Output – 1(if only when present state is S1 and input is 1, otherwise 0).

State – S0, S1

State Transition diagram :



State Table :

Present State	Input (X)	Next state (Q+)	Flip flop input (D)	Output (Y)
0	0	0	0	0
0	1	1	1	0
1	0	1	1	0
1	1	0	0	1

$D = Q^+$ (Characteristic)

$D = f(Q,X)$

$Y = g(Q,X)$

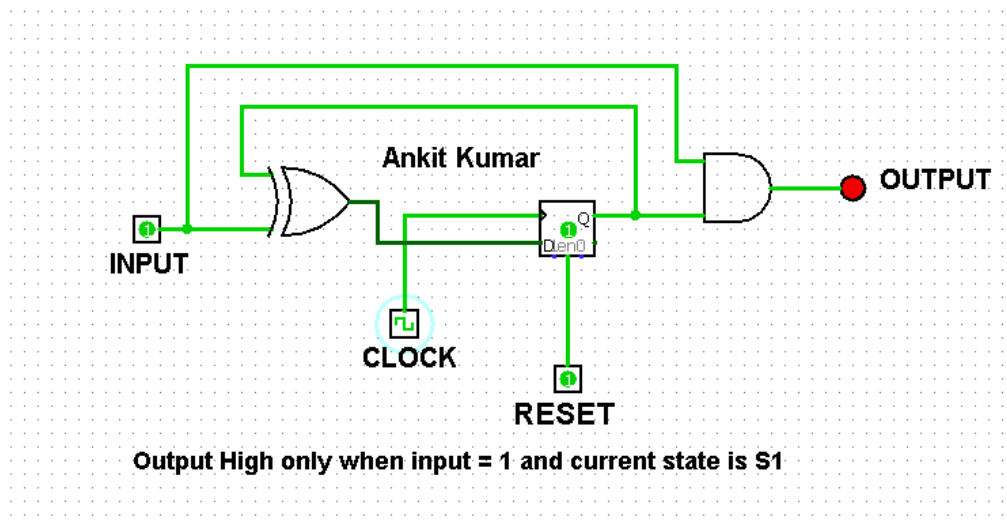
$D = \sum m(1,2)$

$Y = \sum m(3)$

Boolean Function : $D = Q'X + QX' = Q \oplus X$

$Y = QX$

Circuit Diagram :



Verilog : -

```
module mealy_fsm (  
    input clk,  
    input reset,  
    input X,  
    output reg Y  
);  
  
    reg state;  
  
    // State encoding  
    parameter S0 = 1'b0;  
    parameter S1 = 1'b1;  
  
    // State transition logic  
    always @(posedge clk) begin  
        if (reset)  
            state <= S0;  
        else begin  
            case (state)  
  
                S0: begin  
                    if (X == 1'b0)  
                        state <= S0;  
                    else  
                        state <= S1;  
                end  
  
                S1: begin  
                    if (X == 1'b0)  
                        state <= S1;  
                end  
            endcase  
        end  
    end  
    Y = state;
```

```

        else
            state <= S0;
        end

        default: state <= S0;

    endcase
end
end

// Mealy output logic
always @(*) begin
    case (state)

        S0: Y = 1'b0;

        S1: begin
            if (X == 1'b1)
                Y = 1'b1;
            else
                Y = 1'b0;
            end

        default: Y = 1'b0;

    endcase
end

endmodule

```

Verilog Testbench :

```

`timescale 1ns/1ps
module tb_mealy_fsm;

    reg clk;
    reg reset;
    reg X;
    wire Y;

    // Instantiate the FSM
    mealy_fsm uut (
        .clk(clk),
        .reset(reset),
        .X(X),
        .Y(Y)
    );

    // Clock generation

```

```

always #5 clk = ~clk;
initial begin

    // Initialize
    clk = 0;
    reset = 1;
    X = 0;

    // Reset FSM
    #10;
    reset = 0;

    // S0, X=0 → S0, Y=0
    X = 0;
    #10;

    // S0, X=1 → S1, Y=0
    X = 1;
    #10;

    // S1, X=0 → S1, Y=0
    X = 0;
    #10;

    // S1, X=1 → S0, Y=1
    X = 1;
    #10;

    // Test again
    X = 0;
    #10;

    X = 1;
    #10;

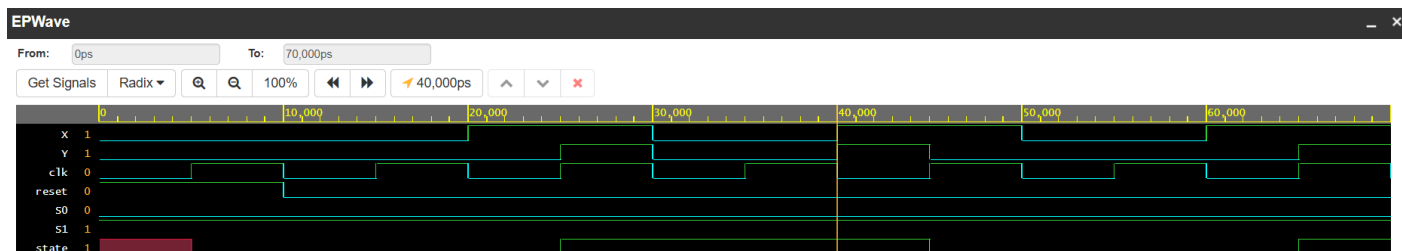
    $finish;
end
// Display simulation results
initial begin
    $monitor("Time=%0t | Reset=%b | State=%b | X=%b | Y=%b",
        $time, reset, uut.state, X, Y);
end
// Generate VCD waveform
initial begin
    $dumpfile("mealy_fsm.vcd");
    $dumpvars(0, tb_mealy_fsm);
end
endmodule

```

Output :

```
VCD info: dumpfile mealy_fsm.vcd opened for output.  
Time=0 | Reset=1 | State=x | X=0 | Y=0  
Time=5000 | Reset=1 | State=0 | X=0 | Y=0  
Time=10000 | Reset=0 | State=0 | X=0 | Y=0  
Time=20000 | Reset=0 | State=0 | X=1 | Y=0  
Time=25000 | Reset=0 | State=1 | X=1 | Y=1  
Time=30000 | Reset=0 | State=1 | X=0 | Y=0  
Time=40000 | Reset=0 | State=1 | X=1 | Y=1  
Time=45000 | Reset=0 | State=0 | X=1 | Y=0  
Time=50000 | Reset=0 | State=0 | X=0 | Y=0  
Time=60000 | Reset=0 | State=0 | X=1 | Y=0  
Time=65000 | Reset=0 | State=1 | X=1 | Y=1  
testbench.sv:55: $finish called at 70000 (1ps)
```

Waveform : -



Problem 3 : Traffic light controller

Solution : Traffic light controller that cycles through three lights (RED , GREEN and YELLOW) with fixed timing.

States : The finite state machine (FSM) consists of 3 states corresponding to the three lamps.

Input : 0 or 1, If input is 1 then move on next state otherwise state will be present state.

Machine Type : This is a Moore machine , as the glowing lamp depends only on the current state and not on external inputs.

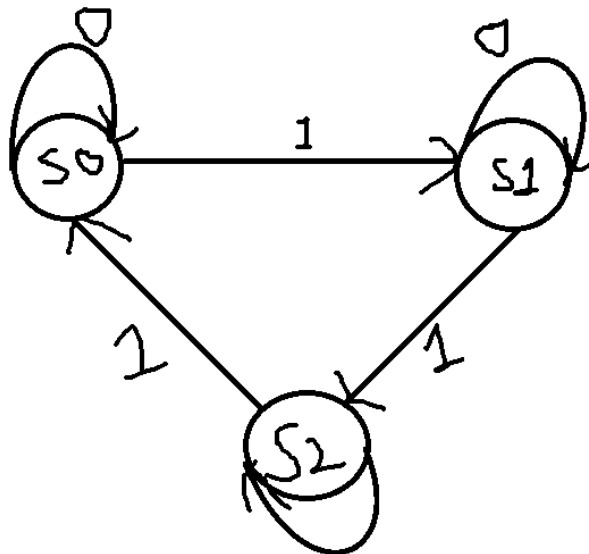
The traffic light has 3 states:

- $S_0 \rightarrow \text{RED}$
- $S_1 \rightarrow \text{GREEN}$
- $S_2 \rightarrow \text{YELLOW}$

Rule:

- If $X = 1 \rightarrow$ move to the next state
- If $X = 0 \rightarrow$ remain in the present state

State Transition Diagram :



State Table :

Present State	Input (X)	Next State	Output
S0	0	S0	Red
S0	1	S1	Red
S1	0	S1	Green
S1	1	S2	Green
S2	0	S2	Yellow
S2	1	S0	Yellow

State Table encoding :

Let : S0 = 00

S1 = 01

S2 = 11.

Current state		Input(X)	Next State		D Flip Flop Input		Output		
Q1	Q0		Q1+	Q0+	D1	D0	Red (Y1)	Green (Y2)	Yellow (Y3)
0	0	0	0	0	0	0	1	0	0
0	0	1	0	1	0	1	1	0	0
0	1	0	0	1	0	1	0	1	0
0	1	1	1	0	1	0	0	1	0
1	0	0	1	0	1	0	0	0	1
1	0	1	0	0	0	0	0	0	1

$D0 = Q1+$ and $D1 = Q0+$ (characteristics of D flip flop)

$D0 = f(Q1, Q0, X)$

$D1 = g(Q1, Q0, X)$

$Y1 = h(Q1, Q0)$

$Y2 = i(Q1, Q0)$

$Y3 = j(Q1, Q0)$

$D1 = \sum m(3, 4)$

$D0 = \sum m(1, 2)$

$R = \sum m(0, 1)$

$G = \sum m(2, 3)$

$Y = \sum m(4, 5)$

D1

	00	01	11	10
0			1	
1	1			

$D1 = Q1 Q0' X' + Q1' Q0 X$

Handwritten truth table for D0:

	$Q_1 X Q_0$	01	11	10
0		1		1
1				

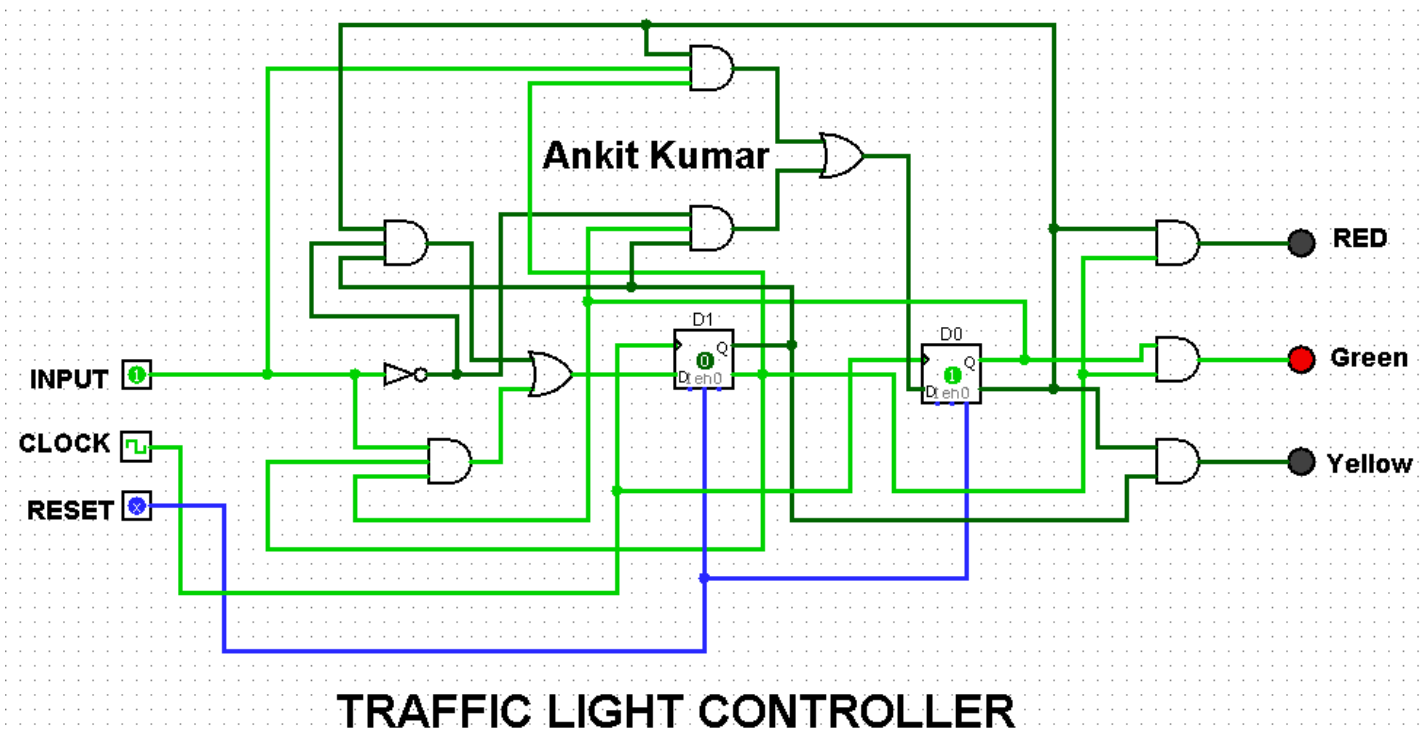
$$D0 = Q1' Q0' X + Q1' Q0 X'$$

$$R = Q1' Q0'$$

$$G = Q1' Q0'$$

$$Y = Q1 Q0'$$

Circuit Diagram of Traffic Light Controller :



Traffic Light Control Verilog :

```
module traffic_light_fsm (
    input clk,
    input reset,
    input X,

    output reg RED,
    output reg GREEN,
    output reg YELLOW
);

// State encoding
parameter S0 = 2'b00; // RED
parameter S1 = 2'b01; // GREEN
parameter S2 = 2'b10; // YELLOW

reg [1:0] state, next_state;

// State register
always @(posedge clk) begin
    if (reset)
        state <= S0;
    else
        state <= next_state;
end

// Next-state logic
always @(*) begin
    case (state)

        S0: begin
            if (X)
                next_state = S1;
            else
                next_state = S0;
        end

        S1: begin
            if (X)
                next_state = S2;
            else
                next_state = S1;
        end

        S2: begin
            if (X)
                next_state = S0;
            else
```

```

        next_state = S2;
    end

    default:
        next_state = S0;

    endcase
end

// Moore output logic
always @(*) begin
    RED  = 1'b0;
    GREEN = 1'b0;
    YELLOW = 1'b0;

    case (state)

        S0: RED = 1'b1;

        S1: GREEN = 1'b1;

        S2: YELLOW = 1'b1;

        default: RED = 1'b1;

    endcase
end

endmodule

```

Traffic light controller testbench :

```

`timescale 1ns/1ps
module traffic_light_fsm_tb;

    reg clk;
    reg reset;
    reg X;

    wire RED;
    wire GREEN;
    wire YELLOW;

    // Instantiate DUT
    traffic_light_fsm uut (
        .clk(clk),
        .reset(reset),
        .X(X),
        .RED(RED),

```

```

.GREEN(GREEN),
.YELLOW(YELLOW)
);

// Clock generation
always #5 clk = ~clk;

initial begin
    // Initialize
    clk = 0;
    reset = 1;
    X = 0;

    // Waveform dump
    $dumpfile("traffic_light.vcd");
    $dumpvars(0, traffic_light_fsm_tb);

    // Reset
    #10;
    reset = 0;
    // X = 0 → Stay in RED
    #20;
    // X = 1 → RED to GREEN
    X = 1;
    #10;
    // X = 0 → Stay in GREEN
    X = 0;
    #20;
    // X = 1 → GREEN to YELLOW
    X = 1;
    #10;

    // X = 0 → Stay in YELLOW
    X = 0;
    #20;

    // X = 1 → YELLOW to RED
    X = 1;
    #10;
    // Stop simulation
    $finish;
end

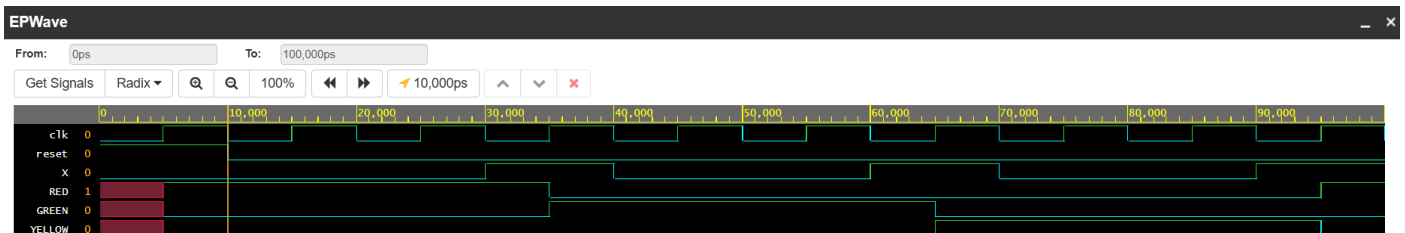
// Display results
initial begin
    $monitor("Time=%0t | Reset=%b | X=%b | RED=%b GREEN=%b YELLOW=%b",
        $time, reset, X, RED, GREEN, YELLOW);
end
endmodule

```

output :

```
VCD info: dumpfile traffic_light.vcd opened for output.  
Time=0 | Reset=1 | X=0 | RED=x GREEN=x YELLOW=x  
Time=5000 | Reset=1 | X=0 | RED=1 GREEN=0 YELLOW=0  
Time=10000 | Reset=0 | X=0 | RED=1 GREEN=0 YELLOW=0  
Time=30000 | Reset=0 | X=1 | RED=1 GREEN=0 YELLOW=0  
Time=35000 | Reset=0 | X=1 | RED=0 GREEN=1 YELLOW=0  
Time=40000 | Reset=0 | X=0 | RED=0 GREEN=1 YELLOW=0  
Time=60000 | Reset=0 | X=1 | RED=0 GREEN=1 YELLOW=0  
Time=65000 | Reset=0 | X=1 | RED=0 GREEN=0 YELLOW=1  
Time=70000 | Reset=0 | X=0 | RED=0 GREEN=0 YELLOW=1  
Time=90000 | Reset=0 | X=1 | RED=0 GREEN=0 YELLOW=1  
Time=95000 | Reset=0 | X=1 | RED=1 GREEN=0 YELLOW=0  
testbench.sv:64: $finish called at 100000 (1ps)
```

Traffic light controller Waveform :



Problem 4: 1011 Sequence Detector

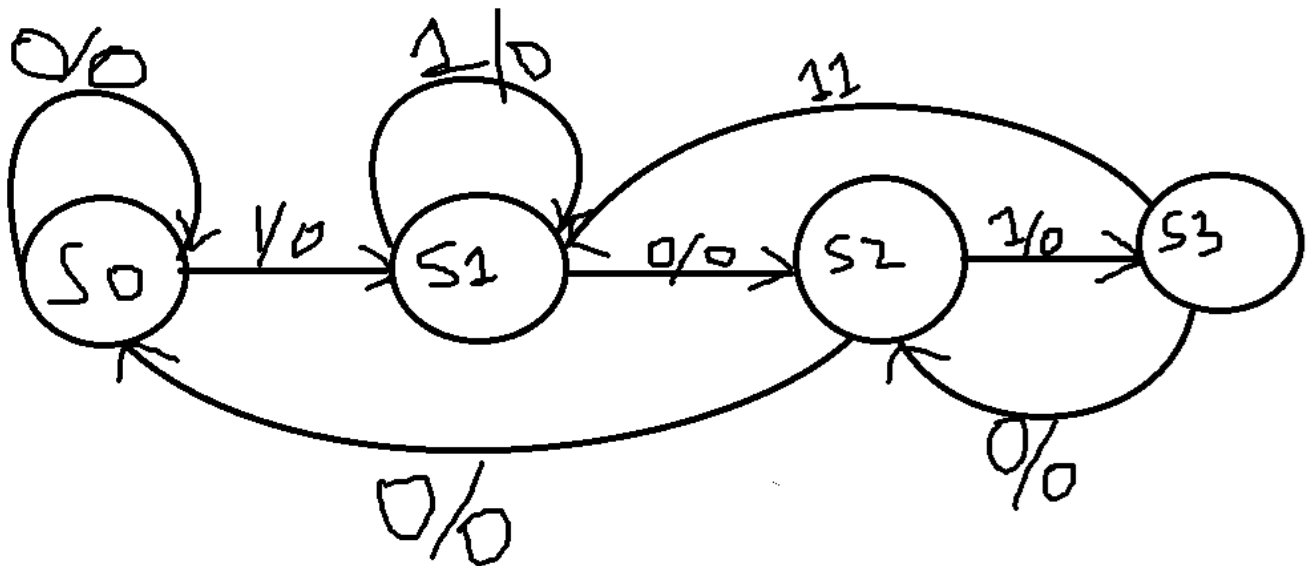
Solution : input = 0, 1

Output = 1 when 1011 detected otherwise output = 0.

State : There are 4 states S0, S1, S2 and S3.

Machine Type : Mealy (Non overlapping)

State Transition Diagram :



State transition Table :

Present State	Next State	
	Input X = 1	Input X = 0
S0	S1, 0	S0, 0
S1	S1, 0	S2, 0
S2	S3, 0	S0, 0
S3	S1, 1	S2, 0

State Encoding :

Let

S0 = 00

S1 = 01

S2 = 10

S3 = 11

State table encoding :

Present State		Input	Next State		Flip flop input		Output
Q1	Q0	X	Q1+	Q0+	D1	D0	Y
0	0	0	0	0	0	0	0
0	0	1	0	1	0	1	0
0	1	0	1	0	1	0	0
0	1	1	0	1	0	1	0
1	0	0	0	0	0	0	0
1	0	1	1	1	1	1	0
1	1	0	1	0	1	0	0
1	1	1	0	1	0	1	1

$$D0 = \sum m(1,3,5,7)$$

$$D1 = \sum m(2,5,6)$$

$$Y = \sum m(7)$$

Simplification using K map :

D0

	00	01	11	10
0		1	1	
1		1	1	

$$D0 = X$$

D1

	00	01	11	10
0				1
1		1		1

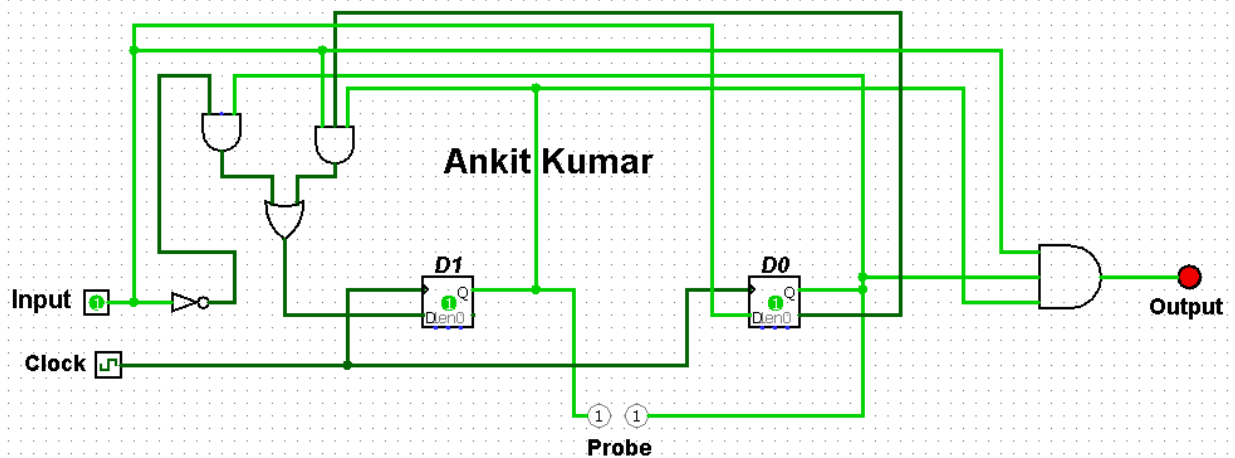
$$D1 = Q0 X' + Q1 Q0 X$$

$$D0 = X$$

$$D1 = Q0 X' + Q1 Q0' X$$

$$Y = Q1 Q0 X$$

Logic circuit :



1011 Sequence Detector

Verilog Program :

```
module sequence_detector_1011 (
    input wire clk,
    input wire reset,
    input wire X,
    output reg Y
);

    reg Q1, Q0;
    reg D1, D0;

    // Next-state logic
    always @(*) begin
        D1 = (Q0 & ~X) | (Q1 & ~Q0 & X);
        D0 = X;
    end

    // State register
    always @(posedge clk) begin
        if (reset) begin
            Q1 <= 1'b0;
            Q0 <= 1'b0;
        end
        else begin
            Q1 <= D1;
            Q0 <= D0;
        end
    end
```



```
end
```

```
// Output logic (Mealy)
always @(*) begin
    Y = Q1 & Q0 & X;
end
```

```
endmodule
```

Testbench :

```
`timescale 1ns/1ps
```

```
module tb_sequence_detector_1011;
```

```
    reg clk;
    reg reset;
    reg X;
```

```
    wire Y;
```

```
    // Instantiate the DUT
    sequence_detector_1011 uut (
        .clk(clk),
        .reset(reset),
        .X(X),
        .Y(Y)
    );
```

```
    // Clock generation
    always #5 clk = ~clk;
```

```
    // Generate waveform
    initial begin
        $dumpfile("1011_detector.vcd");
        $dumpvars(0, tb_sequence_detector_1011);
    end
```

```
    // Display values whenever they change
    initial begin
        $monitor("Time=%0t | clk=%b | reset=%b | X=%b | Q1=%b | Q0=%b | Y=%b",
            $time, clk, reset, X, uut.Q1, uut.Q0, Y);
    end
```

```
    // Test sequence
    initial begin
```

```
        // Initial values
        clk = 0;
        reset = 1;
```

```
X    = 0;

// Keep reset active
#10;

// Release reset
reset = 0;

// Input sequence: 1
X = 1;
#10;

// Input sequence: 0
X = 0;
#10;

// Input sequence: 1
X = 1;
#10;

// Input sequence: 1
// 1011 should be detected here
X = 1;
#10;

// Extra inputs for testing
X = 0;
#10;

X = 1;
#10;

X = 1;
#10;

$finish;
end

endmodule
```

output :

```
VCD info: dumpfile 1011_detector.vcd opened for output.
Time=0 | clk=0 | reset=1 | X=0 | Q1=x | Q0=x | Y=0
Time=5000 | clk=1 | reset=1 | X=0 | Q1=0 | Q0=0 | Y=0
Time=10000 | clk=0 | reset=0 | X=1 | Q1=0 | Q0=0 | Y=0
Time=15000 | clk=1 | reset=0 | X=1 | Q1=0 | Q0=1 | Y=0
Time=20000 | clk=0 | reset=0 | X=0 | Q1=0 | Q0=1 | Y=0
Time=25000 | clk=1 | reset=0 | X=0 | Q1=1 | Q0=0 | Y=0
Time=30000 | clk=0 | reset=0 | X=1 | Q1=1 | Q0=0 | Y=0
Time=35000 | clk=1 | reset=0 | X=1 | Q1=1 | Q0=1 | Y=1
Time=40000 | clk=0 | reset=0 | X=1 | Q1=1 | Q0=1 | Y=1
Time=45000 | clk=1 | reset=0 | X=1 | Q1=0 | Q0=1 | Y=0
Time=50000 | clk=0 | reset=0 | X=0 | Q1=0 | Q0=1 | Y=0
Time=55000 | clk=1 | reset=0 | X=0 | Q1=1 | Q0=0 | Y=0
Time=60000 | clk=0 | reset=0 | X=1 | Q1=1 | Q0=0 | Y=0
Time=65000 | clk=1 | reset=0 | X=1 | Q1=1 | Q0=1 | Y=1
Time=70000 | clk=0 | reset=0 | X=1 | Q1=1 | Q0=1 | Y=1
Time=75000 | clk=1 | reset=0 | X=1 | Q1=0 | Q0=1 | Y=0
testbench.sv:75: $finish called at 80000 (1ps)
Time=80000 | clk=0 | reset=0 | X=1 | Q1=0 | Q0=1 | Y=0
```

Waveform :

